

第十章 多维数组

选自C Primer Plus 10.2



1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

2. 多维数组的应用（设计数据结构）



1.二维数组的表达对象

- 案例：气象分析员要分析近5年中每月的降水量数据，求每年的平均降水量，以及这5年每月的平均降水量。
- 问题：如何表示这些数据？
 - 用60个变量？
 - 用5个一维数组？
 - 可是，如果要分析50年的数据，又该怎么办？
 - 使用一个元素为数组的数组(二维数组) ✓
 - `float rains[5][12];`

1.二维数组的表达对象

- 理解float rains[5][12]:
 - float **rains**[5][12]: rains是一个包含5个元素的数组
 - **float** rains[5][**12**]: rains中每个元素的类型是float [12]
 - 即: rains具有5个元素, 每个元素是包含12个float数值的数组。如: rains[0]是一个数组, 其元素为: rains[0][0],rains[0][1],.....rains[0][11].

rains[0]	rains[0][0]	rains[0][1]		rains[0][11]	
rains[1]	rains[1][0]	rains[1][1]		rains[1][11]	
rains[2]					

1.二维数组的表达对象

- 工程中经常要使用矩阵来表达数据，进行运算

$$\begin{pmatrix} 10, & 23, & 45 \\ 26, & 88, & 39 \\ 77, & 53, & 68 \end{pmatrix} \quad \begin{matrix} & \text{A村} & \text{B村} & \text{C村} \\ \text{A村} & \begin{pmatrix} 0, & 23, & 45 \end{pmatrix} \\ \text{B村} & \begin{pmatrix} 23, & 0, & 39 \end{pmatrix} \\ \text{C村} & \begin{pmatrix} 45, & 39, & 0 \end{pmatrix} \end{matrix}$$

- 二维数组也可以看作矩阵。矩阵中元素类型相同，元素通过所在行和列进行标识。

1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

2. 多维数组的应用（设计数据结构）



2.二维数组的定义和初始化

一、二维数组的定义

类型说明符 数组名[常量表达式1][常量表达式2];

常量表达式1: 表示第一维下标的长度, 运算结果为整型

常量表达式2: 表示第二维下标的长度, 运算结果为整型

例如: `int a[3][4];` 定义了一个3行4列的数组, 数组名为a。该数组共有3*4个整型元素, 用于存储3行4列的矩阵。或者看作3个一维数组的集合。

	第0列	第1列	第2列	第3列
第0行				
第1行				
第2行				

2.二维数组的定义和初始化

二、二维数组元素的表示方法

二维数组的元素也称为**双下标变量**，表示形式为：
数组名[行下标][列下标]

下标应为整型常量、整型变量或整型表达式。

例如：a[2][1] 表示a数组第2行第1列的元素。

	第0列	第1列	第2列	第3列
第0行	a[0][0]	a[0][1]	a[0][2]	a[0][3]
第1行	a[1][0]	a[1][1]	a[1][2]	a[1][3]
第2行	a[2][0]	a[2][1]	a[2][2]	a[2][3]

数组名 ———— 列下标
 行下标

2.二维数组的定义和初始化

三、二维数组的初始化

在定义时对二维数组进行初始化有两种方式：**按行分段赋值**和**按行连续赋值**。

例如对数组a[5][3]:

1. 按行**分段赋值**可写为

```
int a[5][3]={ {80, 75, 92}, {61, 65, 71}, {59, 63, 70},  
              {85, 87, 90}, {76, 77, 85} };
```

2. 按行**连续赋值**可写为

```
int a[5][3]={ 80, 75, 92, 61, 65, 71, 59, 63, 70, 85, 87,  
              90, 76, 77, 85 };
```

这两种赋初值的结果是完全相同的。

1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

2. 多维数组的应用（设计数据结构）



3.二维数组在内存中的存放

- ◆ 实际的存储器是连续编址的，也就是说存储器单元是按一维线性排列的。如何在一维存储器中存放二维数组，可有两种方式：一种是**按行排列**，即放完一行之后顺次放入第二行。另一种是**按列排列**，即放完一列之后再顺次放入第二列。
- ◆ 在C语言中，二维数组是**按行排列**的。按行顺次存放，先存放a[0]行，再存放a[1]行，最后存放a[2]行。每行中有四个元素也是依次存放。

3.二维数组在内存中的存放

a[0]	a[0][0]	
	a[0][1]	
	a[0][2]	
	a[0][3]	
a[1]	a[1][0]	
	a[1][1]	
	a[1][2]	
	a[1][3]	
a[2]	a[2][0]	
	a[2][1]	
	a[2][2]	
	a[2][3]	

```
int a[3][4];
```

第1行第0列元素的地址表示:

```
printf("%p\n", &a[1][0]);
```

```
printf("%p\n", a[1]);
```

以上两种方式是等价的!

二维数组可以看成是一维数组组成的数组, a[1]代表的是一维数组的数组名。

***C语言中二维数组
是按行排列的***

3.二维数组在内存中的存放

$A[i]$ 的内存地址: $A+i*M$ (M 是一个数组元素的字节数)

$A[0]$	$A[1]$	...	$A[i]$	...	$A[\text{ArraySize}-1]$
↑	↑	...	↑	...	↑
A	$A+1*M$	...	$A+i*M$	...	$A+(\text{ArraySize}-1)*M$

因此, 要访问一个一维数组元素, 要知道数组**首地址**、**下标**。

$A[i][j]$ 的内存地址: $A+(i*\text{数组列数}+j)*M$

因此, 要访问一个二维数组元素, 除了要知道数组**首地址**、**行下标**、**列下标**, 还需要知道数组的**列数**。

指针访问 $a[i][j]$: **$*(a+(i*\text{cols}+j))$** ; //不需要加 $*M$

1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

2. 数组的应用（设计数据结构）

3. 指针与多维数组



```
#include <stdio.h>
#define MONTHS 12
#define YEARS 5
int main(void)
{
    const float rains[YEARS][MONTHS] =
    {
        {4.3,4.3,4.3,3.0,2.0,1.2,0.2,0.2,0.4,2.4,3.5,6.6},
        {8.5,8.2,1.2,1.6,2.4,0.0,5.2,0.9,0.3,0.9,1.4,7.3},
        {9.1,8.5,6.7,4.3,2.1,0.8,0.2,0.2,1.1,2.3,6.1,8.4},
        {7.2,9.9,8.4,3.3,1.2,0.8,0.4,0.0,0.6,1.7,4.3,6.2},
        {7.6,5.6,3.8,2.8,3.8,0.2,0.0,0.0,0.0,1.3,2.6,5.2}
    };
};
```

C Primer Plus 252页

```
int year, month;
float subtot, total;
printf(" YEAR      RAINFALL  (inches)\n");
for (year = 0, total = 0; year < YEARS; year++)
{
    // 对于每一年，各月的降水量总和
    for (month = 0, subtot = 0; month < MONTHS; month++)
        subtot += rain[year][month];
    printf("%5d %15.1f\n", 2000 + year, subtot);
    total += subtot; // 所有年度总降水量
}
```



```
for (month = 0; month < MONTHS; month++)
{
    // 对于每个月，各年该月的总降水量
    for (year = 0, subtot =0; year < YEARS; year++)
        subtot += rain[year][month];
    printf("%4.1f ", subtot/YEARS);
}
printf("\n");

return 0;
}
```

1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

2. 多维数组的应用（设计数据结构）



5.二维数组名作函数实参

问题：定义一个函数printMatrix，用于输出二维数组。

printMatrix参数设计考虑

- 二维整型数组元素matrix[i][j]的内存地址：

matrix+(i*列数+j)*sizeof(int);

编译器为了确定matrix[i][j]在内存的位置，必须知道二维数组的列数。方法：二维数组名做形参，第2个下标必须给出,如**int matrix[][10]**。

- 要访问的元素范围：行范围和列范围作为参数传入。
- 因此，函数要访问二维数组，至少需要设计3个参数：
返回值类型 函数名(int matrix[][10],int rows,int cols,...)

打印整数二维数组子函数

```
void printMatrix(int matrix[][COLS],int  
    rows,int cols)
```

二维数组做形参，第2个下标必须给出，
使编译器能确定元素内存地址

```
{  
    int i, j;  
    for (i=0; i<rows; i++){  
        for(j=0; j<cols; j++){  
            printf("%d",matrix[i][j]);  
            putchar(j==cols-1?'\n':' ');  
        }  
    }  
}
```

打印小数二维数组子函数

```
void printMatrix2(float matrix[][COLS],int
    rows,int cols)
{
    int i, j;
    for (i=0; i<rows; i++){
        for(j=0; j<cols; j++)
            printf("%.2f\t",matrix[i][j]);
        printf("\n");
    }
}
```

```
#include <stdio.h>
#define COLS 12
#define ROWS 5
void printMatrix2(float matrix[][COLS],int rows,int cols);
int main(void){
    const float rains[ROWS][COLS] =
    {
        {4.3,4.3,4.3,3.0,2.0,1.2,0.2,0.2,0.4,2.4,3.5,6.6},
        {8.5,8.2,1.2,1.6,2.4,0.0,5.2,0.9,0.3,0.9,1.4,7.3},
        {9.1,8.5,6.7,4.3,2.1,0.8,0.2,0.2,1.1,2.3,6.1,8.4},
        {7.2,9.9,8.4,3.3,1.2,0.8,0.4,0.0,0.6,1.7,4.3,6.2},
        {7.6,5.6,3.8,2.8,3.8,0.2,0.0,0.0,0.0,1.3,2.6,5.2}
    };
    printMatrix2(rains,ROWS,COLS);
}
```

5.二维数组名作函数实参

函数调用: `printMatrix(matrix,10,10);`

只需给出二维数组名，传递的是数组第一个元素的地址

多维数组做函数形参时，需要给出除了第1个下标之外的其他所有的下标

5.二维数组名作函数实参

- 思考：如果要输出的可能只是二维数组的一部分元素，函数怎么设计？

```
void printMatrix(int matrix[][COLS],  
                 int fromRow, int toRow,  
                 int fromCol, int toCol)
```


1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

2. 数组的应用（设计数据结构）

3. 指针与多维数组



矩阵的整体处理：求最大

//求矩阵所有元素中的最大

```
int maxOfMatrix (int matrix[][COLS],int  
    rows,int cols)  
{  
    int i,j,max=matrix[0][0];  
  
    for(i=0; i < rows; i++)  
        for(j=0; j < cols; j++)  
            if(matrix[i][j] > max)  
                max=matrix[i][j];  
  
    return max;  
}
```

矩阵的行处理：求行平均

```
float averageOfRow(int theRow[],int cols)
{
    int i,total=0;

    for(i=0; i < cols; i++)
        total += theRow[i];

    return (float)total/cols;
}
```


函数调用：

```
printf("%.2f\n",averageOfRow(matrix[i],COLS));
```

```
#include <stdio.h>
#define COLS 12
#define ROWS 5
float maxOfMatrix2(float matrix[][COLS],int,int);
float averageOfRow2(float theRow[],int cols);
int main(void){
    const float rains[ROWS][COLS]={...};
    printf("max=%.1f\n",maxOfMatrix2(rain,ROWS,COLS));
    int i;
    for (i=0;i<ROWS;i++)
        printf("%d year,aver=%.1f\n",i+1,
averageOfRow2(rains[i],COLS));
    return 0;
}
```

矩阵的列处理：求列平均

```
float averageOfCol(int matrix[][COLS],int  
    rows,int theCol)  
{  
    int i,total;  
    for(i=0,total=0; i < rows; i++)  
        total += matrix[i][theCol];  
  
    return (float)total/rows;  
}
```

```
#include <stdio.h>
#define COLS 12
#define ROWS 5
float averageOfCol2(float matrix[][COLS],int rows,int
theCol);
int main(void){
    const float rains[ROWS][COLS]={...};
    int i;
    for (i=0;i<COLS;i++)
        printf("%d
month,aver=%.1f\n",i+1,averageOfCol2(rains,ROWS,i));
    return 0;
}
```

例2.矩阵相乘

例2：有M1行和N1列的矩阵A，以及有M2行和N2列的矩阵B。要求计算并输出这两个矩阵的相乘结果。

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \end{bmatrix} \times \begin{bmatrix} 4 & 3 & 2 \\ 3 & 4 & 1 \\ 2 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 8 & 6 \\ 20 & 16 & 12 \\ 30 & 24 & 18 \end{bmatrix}$$

分析： 1.矩阵A要和矩阵B相乘，要求矩阵A的列数要等于矩阵B的行数。相乘结果是一个M1行N2列的矩阵。

2.假设结果矩阵名为result，则

$$\text{result}[\text{row}][\text{col}] = \sum_{i=0}^{\text{矩阵A行数}-1} \text{矩阵A}[\text{row}][i] * \text{矩阵B}[i][\text{col}]$$

```
void inputMatrix1(int matrix[][COL1],int rows,int cols)
{
    int i,j;

    for(i = 0; i < rows; i++)
        for(j = 0; j < cols; j++)
            scanf("%d",&matrix[i][j]);
}

void inputMatrix2(int matrix[][COL2],int rows,int cols)
{
    int i,j;

    for(i = 0; i < rows; i++)
        for(j = 0; j < cols; j++)
            scanf("%d",&matrix[i][j]);
}
```



```
void matrixMultiply(int matrix1[][COL1],int rows1,int  
    cols1,int matrix2[][COL2],int cols2,  
    int result[][COL2])  
{  
    int i,j,k;  
    for(i = 0; i < rows1; i++)  
        for(j = 0; j < cols2; j++){  
            result[i][j] = 0;  
            for(k = 0; k < cols1; k++)  
                result[i][j] += matrix1[i][k] * matrix2[k][j];  
        }  
}
```

矩阵相乘:测试主程序

```
#include <stdio.h>
#define ROW1 3
#define COL1 4
#define ROW2 4
#define COL2 3
void matrixMultiply(int matrix1[][COL1],int,int,int
    matrix2[][COL2],int,int result[][COL2]);
void inputMatrix1(int matrix[][COL1],int,int);
void inputMatrix2(int matrix[][COL2],int,int);
void printMatrix3(int matrix[][COL2],int,int);
int main(void){
    int matrix1[ROW1][COL1], matrix2[ROW2][COL2],result[ROW1][COL2];
    printf("Input matrix 1(%d rows %d cols):\n",ROW1,COL1);
    inputMatrix1(matrix1,ROW1,COL1);
    printf("Input matrix 2(%d rows %d cols):\n",ROW2,COL2);
    inputMatrix2(matrix2,ROW2,COL2);
    matrixMultiply(matrix1,ROW1,COL1,matrix2,COL2,result);
    printf("matrix1*matrix2=\n");
    printMatrix3(result,ROW1,COL2);
    return 0;
}
```

例3：找矩阵鞍点

例3：找矩阵 $[M*N]$ 中的鞍点。即查找矩阵中的元素，该元素为所在行的最大值，并且也是所在列中的最小值。

鞍点

{34,56,23,15,66}

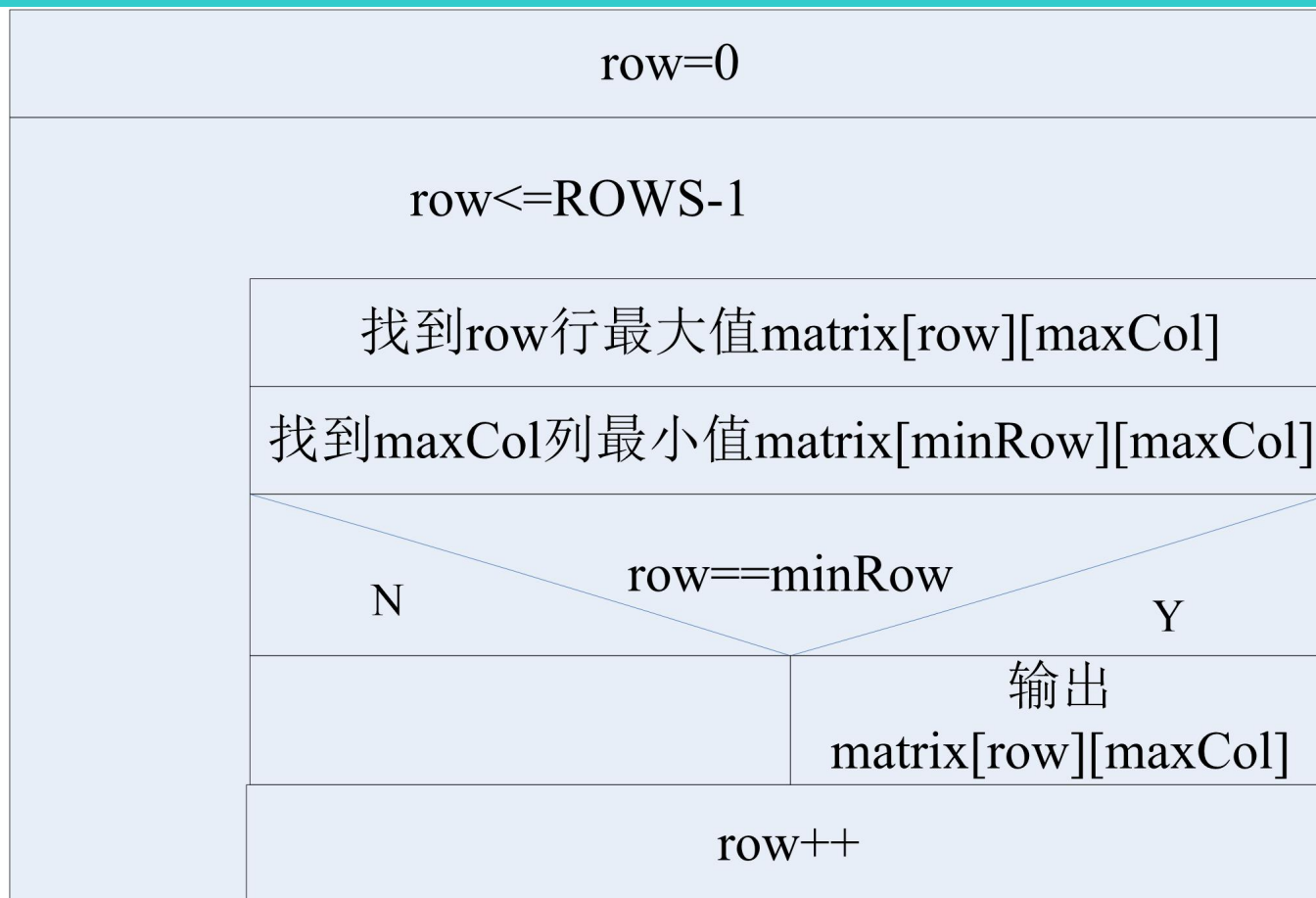
{16,**54**,24,45,36}

{75,83,35,67,28}

{92,59,32,35,18}

{44,55,33,88,77}

例3：找矩阵鞍点



- 设计要点：设计函数findRowMax,返回某行最大元素所在的列号；设计函数findColMin, 返回某列最小元素所在行号；

找行最大值位置的子程序

/* 函数功能：找数组中的最大值元素，并返回其下标

参数说明：数组名，数组中已存放元素的个数*/

调用语句：findRowMax(matrix[i],cols);

```
int findRowMax(int theRow[], int cols)
```

```
{
```

```
    int j, maxLoc=0;
```

```
    for (j=1; j< cols; j++)
```

```
        if (theRow[j] > theRow[maxLoc])
```

```
            maxLoc = j;
```

```
    return maxLoc;
```

```
}
```

找列最小值位置的子程序

/* 函数功能：求col列最小值所在下标并返回

参数说明：数组名，数组行数，要查找的列*/

调用语句：findColMin(matrix,rows,i);

```
int findColMin(int matrix[][COLS], int rows,int theCol)
{
    int i,minRow=0;

    for (i=1; i < rows; i++)
        if (matrix[i][theCol] < matrix[minRow][theCol])
            minRow = i;

    return minRow;
}
```

```
void inputMatrix(int matrix[][COLS],int
    rows,int cols)
{
    int i,j;

    for(i = 0; i < rows; i++)
        for(j = 0; j < cols; j++)
            scanf("%d",&matrix[i][j]);
}
```

找矩阵鞍点的主程序(1)

```
#include <stdio.h>
#define ROWS 100
#define COLS 100
void inputMatrix(int matrix[][COLS],int rows,int cols);
int findRowMax(int theRow[], int rows);
int findColMin(int matrix[][COLS],int,int);

int main(void)
{
    int matrix[ROWS][COLS]; //将矩阵开到最大
    int found;
    int i,j,minRow,maxCol;
    int n,m;
    scanf("%d%d",&n,&m);
    inputMatrix(matrix,n,m); //根据需要设定行、列边界
```


找矩阵鞍点的主程序(2)

```
/* 查找鞍点*/
for(i=0,found=0; i < n; i++){
    maxCol=findRowMax(matrix[i],m);
    minRow=findColMin(matrix,n,maxCol);
    if (minRow==i){ //find saddle point
        printf("The saddle point is
(%d,%d)=%d.\n",i,maxCol,matrix[i][maxCol]);
        found=1;
    }
}
if (found==0)
    printf("There is no saddle point in the matrix.\n");
return 0;
}
```

1. 二维数组

1. 逻辑表达（抽象）
2. 二维数组的定义和初始化
3. 二维数组在内存中的存放
4. 二维数组的逐元素访问
5. 二维数组名做函数实参
6. 二维数组的用于矩阵计算

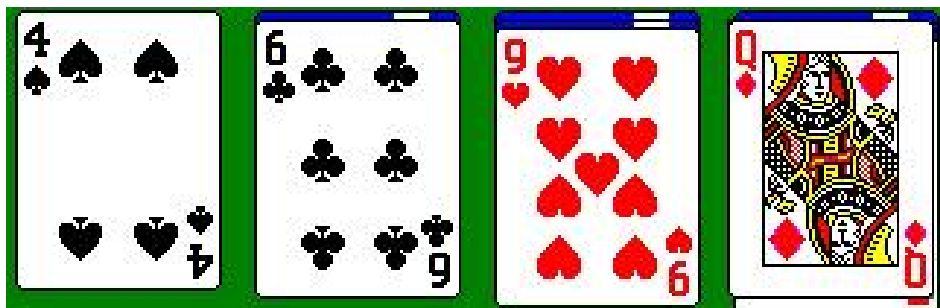
2. 多维数组的应用（设计数据结构）



实例研究一：（模拟发牌和洗牌）

■ 问题描述

每一付牌共有52张（不考虑大小王），
有13种面值，每1种面值 有4种花色。



■ 思考：

1. 计算机如何表示？（抽象）
2. 如何存储？
3. 如何模拟洗牌与发牌？

实例研究一：（模拟发牌和洗牌）

扑克牌的逻辑表达：采用矩阵进行抽象

行与花色对应：分别代表红心、方块、草花、黑心

列代表牌的面值：第0~9列代表A~10

第10~12列代表J、K和Q

数组元素的值表示该张牌在整副牌中的序号（洗牌）。

		A	2	3	4	5	6	7	8	9	10	J	Q	K
		0	1	2	3	4	5	6	7	8	9	10	11	12
红心	0													
方块	1													
草花	2													
黑心	3													

第1张牌是草花K

1

实例研究一：（模拟发牌和洗牌）

扑克牌的存储：每张牌有**花色**、**面值**以及在一副牌中的**序号**这3个信息，考虑到二维数组每个元素有**行下标**、**列下标**以及**元素值**3个信息，因此可以使用4×13的数组deck来存储一副牌，每个数组元素存储一张牌，牌的花色、面值分别由行下标和列下标确定，数组元素值就是牌的序号。

		A	2	3	4	5	6	7	8	9	10	J	Q	K
		0	1	2	3	4	5	6	7	8	9	10	11	12
红心	0													
方块	1													
草花	2													
黑心	3													

实例研究一：（模拟发牌和洗牌）

洗牌的过程

就是将1~52个整数分配到deck数组的52个元素中。

发第*i*张牌：随机地从0~3中选择一行(row)，从0~12中选择一列(column)，把数值*i*插入到deck[row][column]中。

		A	2	3	4	5	6	7	8	9	10	J	Q	K
		0	1	2	3	4	5	6	7	8	9	10	11	12
红心	0													
方块	1										3			
草花	2					1		2						
黑心	3		5								4			

表示洗出的第
1张牌是**草花5**

表示洗出的第**4**
张牌是**黑心10**

实例研究一：（模拟发牌和洗牌）

发牌的过程

就是从deck数组中依次找到值为1、2、3、...52的元素，打印出该元素对应的花色和面值。

		A	2	3	4	5	6	7	8	9	10	J	Q	K
		0	1	2	3	4	5	6	7	8	9	10	11	12
红心	0													
方块	1										3			
草花	2					1		2						
黑心	3		5								4			

发牌结果：发出的第1张牌是草花5，第二张牌是草花7，第3张牌是方块10.....

实例研究一：（模拟发牌和洗牌）

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void shuffle (int [][][13]);
void deal(const int [][][13]);
void outputCard(int i,int row,int col);

int main(void)
{
    int deck[4][13]={0}; //保存一副牌
    srand ((unsigned)time(NULL));
    shuffle(deck);//洗牌
    deal(deck);//发牌

    return 0;
}
```


实例研究一：（模拟发牌和洗牌）

```
void shuffle(int wdeck[][13]) /*洗牌*/
{
    int card,i,j;
    for(card = 1; card <= 52; card++){
        //生成第card张牌的花色和面值
        i = rand()%4;
        j = rand()%13;
        while(wdeck[i][j] != 0){
            i = rand()%4;
            j = rand()%13;
        }
        //确定第card张牌
        wdeck[i][j] = card;
    }
}
```

实例研究一：（模拟发牌和洗牌）

```
void deal (const int wdeck[][13]) //发牌
{
    int card,i,j,found;

    for (card = 1;card <= 52;card++){
        //遍历二维数组，查找值为card的元素，输出对应的花色和面值
        found = 0;
        for (i=0; i < 4 && !found; i++)
            for (j=0; j < 13 && !found; j++)
                if (wdeck[i][j]==card){
                    found = 1;
                    outputCard(card,i,j);
                    //输出第card张牌的花色面值
                }
    }
}
```

实例研究一：（模拟发牌和洗牌）

```
void outputCard(int no,int row,int
    col)
{
    printf("No %d card:",no);
    switch (col){
        case 0: printf("Acer");break;
        case 1: printf("Deuce");break;
        case 2: printf("Three");break;
        case 3: printf("Four");break;
        case 4: printf("Five");break;
        case 5: printf("Six");break;
        case 6: printf("Seven");break;
        case 7: printf("Eight");break;
        case 8: printf("Nine");break;
        case 9: printf("Ten");break;
        case 10: printf("Jack");break;
        case 11: printf("Queen");break;
        case 12: printf("King");break;
        default:
            printf("invalid col!");
    }
}
```

```
    printf(" of ");
    switch (row){
        case
0: printf("Hearts");break;
        case
1: printf("diamonds");break;
        case
2: printf("Clubs");break;
        case
3: printf("Spades");break;
        default: printf("invalid
row!");
    }
    printf("\n");
}
```

■ 问题描述

设想有一只机械海龟，他在程序控制下在屋里四处爬行。海龟拿了一只笔，这支笔或者朝上，或者朝下。当笔朝下时，海龟用笔画下自己的移动轨迹；当比朝上时，海龟在移动过程中什么也不画。

海龟会读取一系列操作命令（程序）。按照程序的要求完成各种动作。

假定海龟总是从地板上（0,0）出发，并且开始时笔是朝上的。

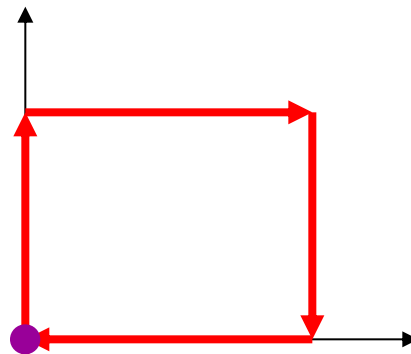
实例研究二：海龟作图

海龟接收到的命令（指令集）

命令	含义
1	笔朝上
2	笔朝下
3	右转弯
4	左转弯
5, 10	向前走10格(或其他格数)
6	打印50×50的数组
9	数据结束（标记）

操作(指令)序列

初始状态：
笔朝上，
海龟在原
点



根据右侧“程序”
绘制的
12*12矩形

2	
5	12
3	
5	12
3	
5	12
3	
5	12
1	
6	
9	

- **问题分析：**

- 计算模型（状态机）。状态、输入（激励）、状态转移。
- 海龟的状态，依赖于下面三个因素：
 1. 笔的朝向（write=0/1, 分别表示笔朝上和朝下）
 2. 海龟当前位置(坐标row、col)
 3. 海龟的朝向（dir=EAST/SOUTH/WEST/NORTH）
- 输入：接收的命令（指令）序列

设计：（数据结构设计）

- 1、设计一个50*50的数组floor，用于记录海龟绘制的图形，数组元素初始化为0。在海龟爬行过程中，如果笔朝下，就把数组floor中对应于海龟所处位置的元素置为1。

`int floor[50][50]={0};`当在第i行第j列绘制了图，则`floor[i][j]`置为1

- 2、设计一个存放操作命令的数组：

`int commands[100];`

设计：（算法设计）

- 计算过程：在初始状态下，接收命令，状态转移，直至接收到结束命令。
- 在海龟爬行过程中，如果笔朝下，就把数组 floor 中对应于海龟所处位置的元素置为 1。当给出命令 6（打印命令）后，数组中元素为 1 的位置全部用星号显示，元素为 0 的位置全部用空格显示。

实例研究二：海龟作图

```
#define COMMANDSIZE 100
#define PICSIZE 50
#define EAST 0
#define SOUTH 1
#define WEST 2
#define NORTH 3
void printGraph(int matrix[][PICSIZE],int,int);
void draw(int commands[],int,int
    floor[][PICSIZE],int,int);
void getCommand(int commands[],int);
int main(void)
{
    int commands[COMMANDSIZE]; //存储命令
    int floor[PICSIZE][PICSIZE]={0}; //存储图片
    getCommand(commands,COMMANDSIZE); //获取命令
    draw(commands, COMMANDSIZE ,floor, PICSIZE,
PICSIZE); //作图
    return 0;
}
```

实例研究二：海龟作图-获取命令

//简化的命令输入函数，不考虑命令过多导致数组越界问题

```
void getCommand(int commands[],int size)
{ int i=0;
  printf("input the commands\n");
  scanf("%d",&commands[i]);
  while(commands[i]!=9 ){
    if (commands[i]==5){ //继续读取前进的格数
      printf("input steps:");
      i++;
      scanf("%d", &commands[i]);
    }
    i++;
    scanf("%d", &commands[i]);
  }
}
```

实例研究二：海龟作图-作图

```
void draw(int commands[], int size,  
          int floor[][PICSIZE], int rows, int cols);
```

海龟是否作图，以及作图的轨迹依赖于下面三个因素：

1. 笔的朝向（write=0/1, 分别表示笔朝上和朝下）
2. 海龟当前位置(坐标row、col)
3. 海龟的朝向（dir=EAST/SOUTH/WEST/NORTH）

```
i=0;write=0;dir=NORTH;  
row=49,col=0
```

```
commands[i]!=9
```

```
    执行命令commands[i]
```

```
    i++
```

命令1、2：改变笔的朝向write

命令3、4：改变海龟朝向dir
（依赖于当前海龟朝向）

命令5：作图(依赖于海龟的朝向和当前坐标)，改变海龟位坐标(row,col)

命令6：打印floor

```
i=0;write=0;dir=NORTH;  
row=49,col=0
```

```
commands[i]!=9
```

```
执行命令commands[i]
```

```
i++
```

实例研究二：海龟作图-作图

```
switch (commands[i]){  
    case 1:write=0;break;  
    case 2:write=1;break;  
    case 3://右转  
        //修改海龟朝向（北变东，东变南，南变西，西变北）  
    case 4://左转  
        //修改海龟朝向（北变西，西变南，南变东，东变北）  
    case 5://画图  
        if (write==1) //如果笔朝下，则画图  
            //画图，更新海龟坐标(四个朝向分别处理)  
        else //如果笔朝上,则只是简单修改海龟的坐标  
            //更新海龟坐标（四个朝向分别处理）  
    case 6:printArray(floor,50,50);break;  
}
```

```
void draw(int commands[],int size,int floor[]
[PICSIZE],int rows,int cols)
{ int i,j;//循环控制变量
  int row=rows-1,col=0;//海龟初始位置在坐标原点
  int dir=NORTH;//海龟初始方向是向北
  int write=0; //0记录笔朝上； 1： 朝下。
  for (i=0; i<size && commands[i]!=9; i++)
    switch (commands[i]){
      case 1: write=0; break;
      case 2: write=1; break;
```

case 3://右转, 改变海龟朝向

```
if (dir>=0 && dir<4) dir=(dir+1)%4;
```

```
else printf(" error dir\n");
```

```
/*switch (dir){
```

```
    case EAST:    dir=SOUTH; break;
```

```
    case SOUTH:   dir=WEST;  break;
```

```
    case WEST:    dir=NORTH; break;
```

```
    case NORTH:   dir=EAST;  break;
```

```
    default: printf(" error
```

```
dir\n");break;
```

```
    }*/
```

```
break;
```

```
case 4://左转, 改变海龟朝向
    if (dir>=0 && dir<4) dir=(dir+3)%4;
    //dir=(dir-1)>=0?dir-1:3; 但不能是(dir-1)%4;
    else printf(" error dir\n");
    /*switch (dir){
        case EAST:    dir=NORTH; break;
        case SOUTH:   dir=EAST;  break;
        case WEST:    dir=SOUTH; break;
        case NORTH:   dir=WEST;  break;
        default:      printf("error dir\n");break;
    }*/
    break;
```

```
case 5://画图
```

```
    i++; //用于读取前进的步数
```

```
    if (write==1){ //如果笔朝下，则画图
```

```
        for(j=1; j<=commands[i]; j++){
```

```
            switch (dir){
```

```
                case EAST:  col++;break;
```

```
                case SOUTH: row++;break;
```

```
                case WEST:  col--;break;
```

```
                case NORTH: row--;break;
```

```
                default:printf("error dir in case
```

```
5\n");}
```

```
        floor[row][col]=1;
```

```
    }//end for(j)
```

```
}
```


else{//如果笔朝上,则只是简单修改海龟的坐标

```
switch (dir){
```

```
    case EAST: col+=commands[i]; break;
```

```
    case SOUTH:row+=commands[i]; break;
```

```
    case WEST: col-=commands[i]; break;
```

```
    case NORTH:row-=commands[i]; break;
```

```
    default:printf("invalid direction in second
```

```
part of case 5\n");
```

```
    }
```

```
}
```

```
break;
```

```
case 6:printGraph(floor,rows,cols);break;
```

```
//case
```

```
}
```

实例研究二：海龟作图-打印图形

```
void printGraph(int matrix[][PICSIZE],int rows,int cols
{
    int i,j;
    printf("the graph is:\n");
    for(i=0; i < rows; i++){
        for(j=0; j < cols; j++)
            if(matrix[i][j]!=0)
                putchar('*');
            else
                putchar(' ');
        putchar('\n');
    }
}
```

